
Modül 07 — Spring Data MongoDB

Ödevler

Spring Boot Edu



2026-09-23

İçindekiler

Nasıl Çalışılır?	2
Ödev 1 — Değişken Özellikli Ürün Kataloğu (Kolay)	2
Ödev 2 — Kategori Bazlı Gelir Raporu (Orta)	2
Ödev 3 — Ağırlıklı Tam Metin Arama (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	3

Nasıl Çalışılır?

1. Başlangıç kodu modules/07-data-mongodb/exercise/ içindedir. TODO yorumlarını bulun.
2. Testler gerçek bir MongoDB ile çalışır (Testcontainers). **Docker açık olmalıdır.**
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/07-data-mongodb/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takıldığınızda önce ipuçlarına, sonra modules/07-data-mongodb/solution/ içindeki çözüme bakın.

★ İpucu

Ödevler birbirinden bağımsızdır. Her test sınıfı kendi verisini ekler.

Ödev 1 — Değişken Özellikli Ürün Kataloğu (Kolay)

Hedef: Farklı türde ürünleri, şema değiştirmeden aynı koleksiyonda saklamak.

Kitapların sayfa sayısı, çantaların rengi ve malzemesi, kalemlerin rengi var. Hepsi products koleksiyonunda duracak.

Yapılacaklar (exercisel paketi):

- TODO 1a — Product: products koleksiyonunda saklansın ve iki ürün aynı sku'yu paylaşmasın.
- TODO 1b — findByCategoryOrderByPrice: bir kategorinin ürünleri, en ucuzu önce. Türetilmiş sorgu kullanın.
- TODO 1c — findByColor: attributes haritasındaki color değerine göre JSON sorgusu.

İpuçları:

- Ders bölüm 3.1 ve 3.5: @Document("..."), @Indexed(unique = true).
- Ders bölüm 3.2: @Query("{ 'attributes.language': ?0 }").
- default metodu silip yerine soyut bir metot bildirmeniz yeterli.

Kabul kriterleri: Exercise1Test içindeki 4 testin hepsi geçer.

Tahmini süre: 20 dakika

Ödev 2 — Kategori Bazlı Gelir Raporu (Orta)

Hedef: Bir raporu uygulamada değil, aggregation pipeline ile veritabanında hesaplamak.

Yapılacaklar (`exercise2.SalesReport.revenuePerCategory`):

- TODO 2a — Yalnızca `from <= soldAt < until` aralığındaki satışlar.
- TODO 2b — Kategori başına `units = quantity` toplamı, `revenue = quantity * unitPrice` toplamı.
- TODO 2c — `_id`'yi `category` alanına eşleyin ve gelire göre, en yüksek önce sıralayın.

İpuçları:

- Ders bölüm 3.4: `newAggregation(match(...), group(...), project(...), sort(...))`.
- `where("soldAt").gte(from).lt(until)`.
- Çarpımın toplamı: `.sum(ArithmeticOperators.Multiply.valueOf("quantity").multiplyBy("unitPrice"))`.
- Sonucu `record`'a eşlemek için: `mongo.aggregate(pipeline, Sale.class, Category-Revenue.class)`.

Kabul kriterleri: `Exercise2Test` içindeki 2 testin ikisi de geçer.

Tahmini süre: 40 dakika

Ödev 3 — Ağırlıklı Tam Metin Arama (Zor)

Hedef: Başlıkta geçen kelimenin, metin gövdesinde geçenden daha önemli sayıldığı bir arama yapmak.

Yapılacaklar (`exercise3` paketi):

- TODO 3a — `Article`: `title` (ağırlık 3) ve `body` (ağırlık 1) metin index'ine girsin.
- TODO 3b — `ArticleSearch.search`: kelimelerden **herhangi birini** içeren makaleleri, alaka puanına göre sıralı ve en fazla `limit` kadar döndürün.

İpuçları:

- `@TextIndexed(weight = 3)`.
- `TextQuery.queryText(TextCriteria.forDefaultLanguage().matchingAny(...)).sortByScore()` ve `query.limit(...)`.
- Kelimeleri boşluktan bölmek için: `words.split("\\s+").`
- Metin arama kökleri de tanır: "indexes" araması "index" içeren makaleleri bulur.

Kabul kriterleri: `Exercise3Test` içindeki 4 testin hepsi geçer.

Tahmini süre: 40 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Ödev 2'nin raporunu `$facet` ile genişletin: aynı pipeline hem kategori bazlı geliri hem de toplam geliri tek bir sorguda döndürsün. `explain()` ile `soldAt` alanına bir index eklemenin sorguyu nasıl değiştirdiğini inceleyin.